Reinforcement Learning End-to-End Autonomous Racing Architecture with Asymmetric SAC and Sim-to-Real Pipeline

Woobin Choi, Subin Lim, Chaehyeon Jung, Minho Kim, and Juyoung Choi
In The END
Soongsil University

***Abstract*—This report presents a robust autonomous racing architecture that seamlessly integrates a model-free Asymmetric Soft Actor-Critic (SAC) framework with a rigorous three-tier Sim-to-Real pipeline. By leveraging a high-performance 1/8 scale chassis and an automated two-phase curriculum, the proposed system effectively handles both single-vehicle time trials and dynamic multi-vehicle overtaking scenarios without relying on global racelines.**

***Index Terms*—Autonomous Racing, Soft Actor-Critic, Reinforcement Learning, Sim-to-Real, Isaac Sim.**

I. Introduction

High-speed autonomous racing requires a tight coupling between high-fidelity simulation and physical deployment. This technical report details our hardware configuration based on the Serpent SRX8 R chassis and our software stack governed by an Asymmetric Soft Actor-Critic (SAC) algorithm with an automated training curriculum.

II. Hardware Architecture

The autonomous racing vehicle is built upon a high-performance 1/8 scale platform designed to bridge the gap between simulation and physical environments. The core hardware components are summarized below:

- **Chassis:** Serpent SRX8 R (1/8 scale premium racing chassis).
- **Main Computer:** NVIDIA Jetson Orin Nano.
- **Motor & Actuator:** REDS V8 brushless motor with an HV Digital Servo.
- **VESC & IMU:** VESC 6MK VI HP with an integrated IMU.
- **Power System:** LiHV 2S $\times$ 2 battery configuration.

III. Software Stack & Core Algorithms

The software architecture is built on a Model-Free Asymmetric Soft Actor-Critic (SAC) algorithm. It operates reactively and robustly under a progressive Two-Phase Automatic Curriculum Manager.

A. State and Privileged Observation Space

To maximize training efficiency while ensuring deployability, we adopt an asymmetric actor-critic framework:

- **Actor Policy (π):** Receives a compressed, history-stacked observation ($N = 4$) containing a 1/4 downsampled 2D LiDAR scan ($N_{\text{scan}} = 270$) and ego-states $\mathbf{s}_{\text{ego}} = [v_x, \delta_{t-1}, \mathbb{I}_{\text{overtake}}]$. This ensures the policy network relies solely on realistic, onboard sensor data during deployment.
- **Critic Network (Q):** Benefits from an expanded 24-dimensional Privileged Observation Space during the offline training phase. This space includes absolute track coordinates, precise lateral/heading errors ($e_{\text{lat}}, e_{\text{psi}}$), lookahead track curvatures, exact wall distances via pixel-level raycasting, and relative target vehicle coordinates.

B. Two-Phase Automatic Curriculum

To prevent the agent from falling into sub-optimal local minima, the training process is governed by an automated curriculum state-machine:

1) **Phase 1 (Solo Time Trial):** The opponent vehicle is isolated far outside the track bounding box (100m margin) to simulate a pure single-agent environment. The agent focuses on learning curvature-dependent velocity scaling and wall avoidance. Once the Exponential Moving Average (EMA) of the lap completion rate exceeds 60% over a window of 100 episodes (with a minimum of $2,000$ episodes), Phase 2 is triggered.
2) **Phase 2 (Dynamic Overtaking):** The opponent vehicle is spawned ahead with an adaptive gap (initially wide at 30–50 waypoints, progressively closing down to 5–50). The opponent utilizes a Pure Pursuit controller with randomized lookahead profiles (20% aggressive, 20% defensive, 60% nominal) to prevent the ego-agent from over-fitting to a specific opponent profile.

C. Reward Design and Motion Smoothing

The reward function integrates aggressive progress incentives with strict physics-informed constraints:

$$\mathcal{R} = r_{\text{progress}} + r_{\text{safety}} + r_{\text{TTC}} + r_{\text{overtake}} \tag{1}$$

To eliminate high-frequency weaving on straightaways, a speed-scaled penalty is introduced:

$$\mathcal{B}_{\text{steer}} = -|\delta_t - \delta_{t-1}| \cdot \left(\lambda_H + \frac{v_x}{V_{\text{max}}}\lambda_{H\_speed}\right) \tag{2}$$

This dampens rapid steering fluctuations at high velocities while preserving agile cornering capabilities at low speeds.

Furthermore, a Time-To-Collision (TTC) safety net evaluates the time-to-collision relative to the leading opponent:

$$\text{TTC} = \frac{\text{Gap}}{v_{\text{ego}} - v_{\text{opp}}} \tag{3}$$

A reciprocal penalty $1/\text{TTC}$ is applied when a collision is imminent (TTC < 5.0s), prompting the model to learn sophisticated, predictive braking and strategic trailing behavior.

IV. Sim-to-Real Deployment Pipeline

To bridge the reality gap safely and efficiently, the architecture adopts a rigorous three-tier deployment pipeline (Fig. 1).

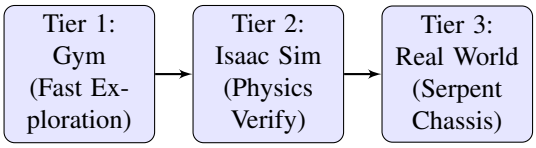

Fig. 1. Three-tier Sim-to-Real deployment pipeline workflow.

1) **Tier 1: F1tenth Gym Environment (Fast Exploration):** Used for highly accelerated parallel training across multiple CPU workers. The core curriculum transitions, reward shaping, and asymmetric features extraction are fully matured here over millions of timesteps.
2) **Tier 2: NVIDIA Isaac Sim (High-Fidelity Physics Verification):** The trained policy is transferred to Isaac Sim to evaluate the agent under realistic multi-body dynamics, surface friction characteristics, and simulated sensor noise. This stage verifies that the LiDAR downsampling ($N_{\text{scan}} = 270$) and state estimations remain stable under vehicle chassis roll, pitch, and high-speed tire slip conditions.
3) **Tier 3: Physical Deployment & Motor Protection:** When deploying the model on the physical Jetson Orin Nano, an Action Low-Pass Filter is activated within the control loop:

$$a_t = (1 - \alpha)a_{t-1} + \alpha a_{\text{model}} \tag{4}$$

By setting $\alpha = 1.0$ during simulation for maximum exploration, it is tuned down (e.g., $\alpha = 0.4 \sim 0.6$) on the real car. This dampens high-frequency actuator commands (jerk), protecting the physical HV Digital Servo and REDS V8 motor drivetrain from mechanical stress while ensuring smooth, human-like racing lines.

V. Conclusion

This architecture successfully demonstrates that model-free reinforcement learning, guided by an asymmetric observation critic and an automated training curriculum, can achieve agile and safe autonomous racing. The combination of a premium 1/8 scale Serpent chassis and low-pass action filtering provides a robust foundation for transferring high-speed overtaking policies from simulation to the real world.